



LeetCode 268 — Missing Number

1. Problem Title & Link

Title: LeetCode 268: Missing Number

Link: <https://leetcode.com/problems/missing-number/>

2. Problem Statement (Short Summary)

You are given an array `nums` containing **n distinct numbers** taken from the range **$[0, n]$** .

👉 Exactly **one number is missing** from the array.

Your task is to **find and return the missing number**.

Constraints require:

- **$O(n)$** time
- **$O(1)$** extra space

3. Examples (Input → Output)

Example 1

Input:

`nums = [3,0,1]`

Output:

2

Example 2

Input:

`nums = [0,1]`

Output:

2

Example 3

Input:

`nums = [9,6,4,2,3,5,7,0,1]`

Output:

8

4. Constraints

- $n == \text{nums.length}$
- $1 \leq n \leq 10^4$
- Numbers are in range $[0, n]$



- Exactly **one number missing**
- Must use **O(1)** extra space

5. Core Concept (Pattern / Topic)

Bit Manipulation — XOR Trick

(Also solvable using Math Sum Formula)

6. Thought Process (Step-by-Step Explanation)

✗ Brute Force

Sort and scan → $O(n \log n)$ ✗

✗ HashSet

Track seen numbers → $O(n)$ space ✗

✓ Optimized — XOR Approach (BEST)

Key XOR properties:

$$x \oplus x = 0$$

$$x \oplus 0 = x$$

Idea:

- XOR all indices from 0 to n
- XOR all values in the array
- All matching numbers cancel out
- The remaining value is the **missing number**

Why this works:

- Array contains all numbers except one
- XOR cancels duplicates automatically

7. Visual / Intuition Diagram (ASCII)

Example:

nums = [3,0,1]

n = 3

XOR indices: $0 \oplus 1 \oplus 2 \oplus 3$

XOR nums: $3 \oplus 0 \oplus 1$



Combine:

$$(0^1 1^2 3) ^ (3^0 1)$$

→ 2

8. Pseudocode (Language Independent)

```
result = n
for i from 0 to n-1:
    result = result XOR i XOR nums[i]
return result
```

9. Code Implementation

✓ Python

```
class Solution:
    def missingNumber(self, nums):
        n = len(nums)
        result = n

        for i in range(n):
            result ^= i ^ nums[i]

        return result
```

✓ Java

```
class Solution {
    public int missingNumber(int[] nums) {
        int n = nums.length;
        int result = n;

        for (int i = 0; i < n; i++) {
            result ^= i ^ nums[i];
        }

        return result;
    }
}
```

10. Time & Space Complexity

Time



- $O(n)$ — single pass

Space

- $O(1)$ — constant extra space

11. Common Mistakes / Edge Cases

✗ Mistakes

- Forgetting to include n in XOR
- Using sum formula without handling overflow
- Sorting unnecessarily

✓ Edge Cases

- Missing number = 0
- Missing number = n
- Single element array

12. Detailed Dry Run (Step-by-Step Table)

Input:

nums = [3,0,1]

i	nums[i]	result
start	—	3
0	3	$3 \oplus 0 \oplus 3 = 0$
1	0	$0 \oplus 1 \oplus 0 = 1$
2	1	$1 \oplus 2 \oplus 1 = 2$

Final Answer = 2

13. Common Use Cases (Real-Life / Interview)

- Detecting missing IDs
- Data integrity checks
- Error detection systems
- Stream data validation

14. Common Traps

- Using sum formula and risking overflow
- Forgetting XOR identity
- Assuming array is sorted

15. Builds To (Related LeetCode Problems)



- LC 136 — Single Number
- LC 137 — Single Number II
- LC 260 — Single Number III
- LC 287 — Find the Duplicate Number

16. Alternate Approaches + Comparison

1 Math Sum Formula

$\text{expected} = n * (n + 1) / 2$

$\text{missing} = \text{expected} - \text{actual_sum}$

- Simple
- Risk of overflow (in other languages)

✓ 2 XOR Method (BEST)

- No overflow
- Constant space
- Interview favorite

17. Why This Solution Works (Short Intuition)

All numbers from 0 to n appear exactly once **except one**.

XOR removes all duplicates and leaves behind the missing number.

18. Variations / Follow-Up Questions

- What if two numbers are missing?
- What if numbers are duplicated?
- What if array contains negatives?
- Can you solve in-place?